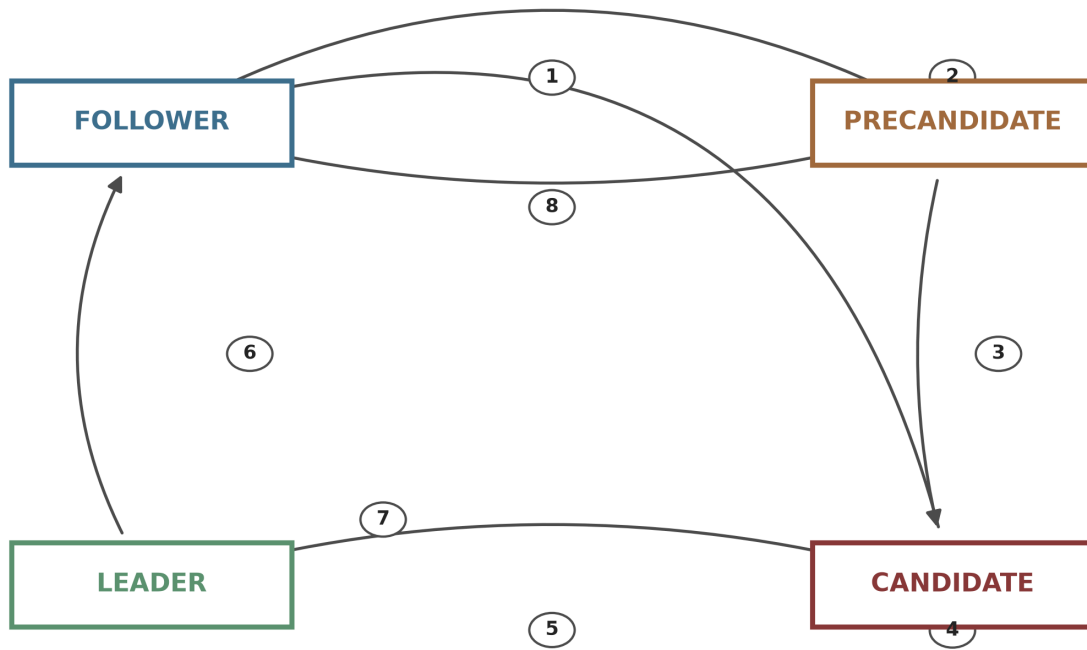


QUORUM-7 per-node role state machine (S03-S05)  
current\_term only increments on winning a pre-vote majority, never directly on timeout



- 1 own effective timeout elapses (election\_elapsed reaches BASE\_TIMEOUT + 20\*node\_id)
- 2 timeout again without a pre-vote majority: fresh pre-vote round, current\_term unchanged
- 3 wins pre-vote majority ( $\geq 3/5$ ): increment current\_term, vote for self, request real votes
- 4 split real vote, timeout again: return to a fresh pre-vote round, current\_term unchanged
- 5 wins real-vote majority ( $\geq 3/5$ ): init next\_index/match\_index per follower, start replication
- 6 (from LEADER) sees an AppendEntries or vote request at a term  $\geq$  its own current\_term
- 7 (from CANDIDATE) sees an AppendEntries at term  $\geq$  own (another node already won this term)
- 8 (from PRECANDIDATE) sees an AppendEntries at term  $\geq$  own while still pre-voting

Figure 1 -- Per-node role state machine. Pre-vote gates every real election.

Bounded-batch AppendEntries with per-follower next\_index/match\_index (S06)  
K=4 entries per round; a lagging follower needs multiple rounds to catch up

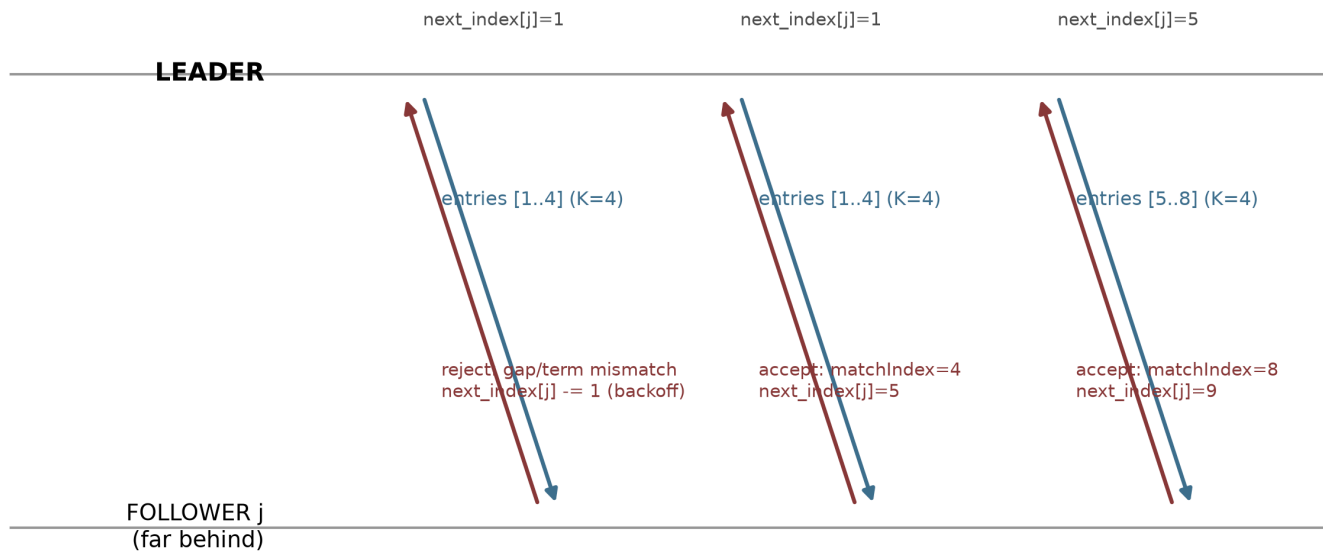


Figure 2 -- Bounded-batch AppendEntries, K=4, per-follower next\_index/match\_index (schematic).

Per-node election-timeout stagger across the 3 swept BASE\_TIMEOUT settings (S03)

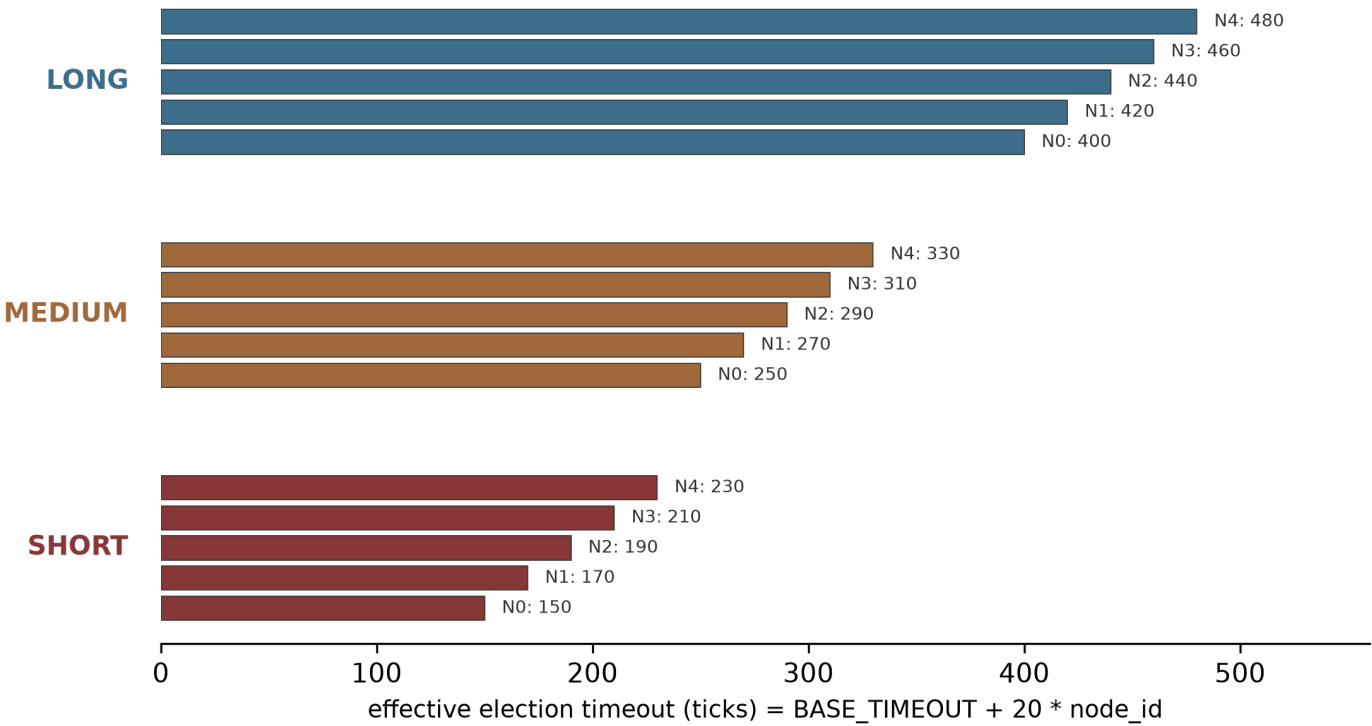


Figure 3 -- Per-node election-timeout stagger across the 3 swept BASE\_TIMEOUT settings.

Per-scenario fault window, offsets relative to the run's own anchor tick (S11)  
each scenario is one continuous 2400-tick run; only the fault window shown here is scripted

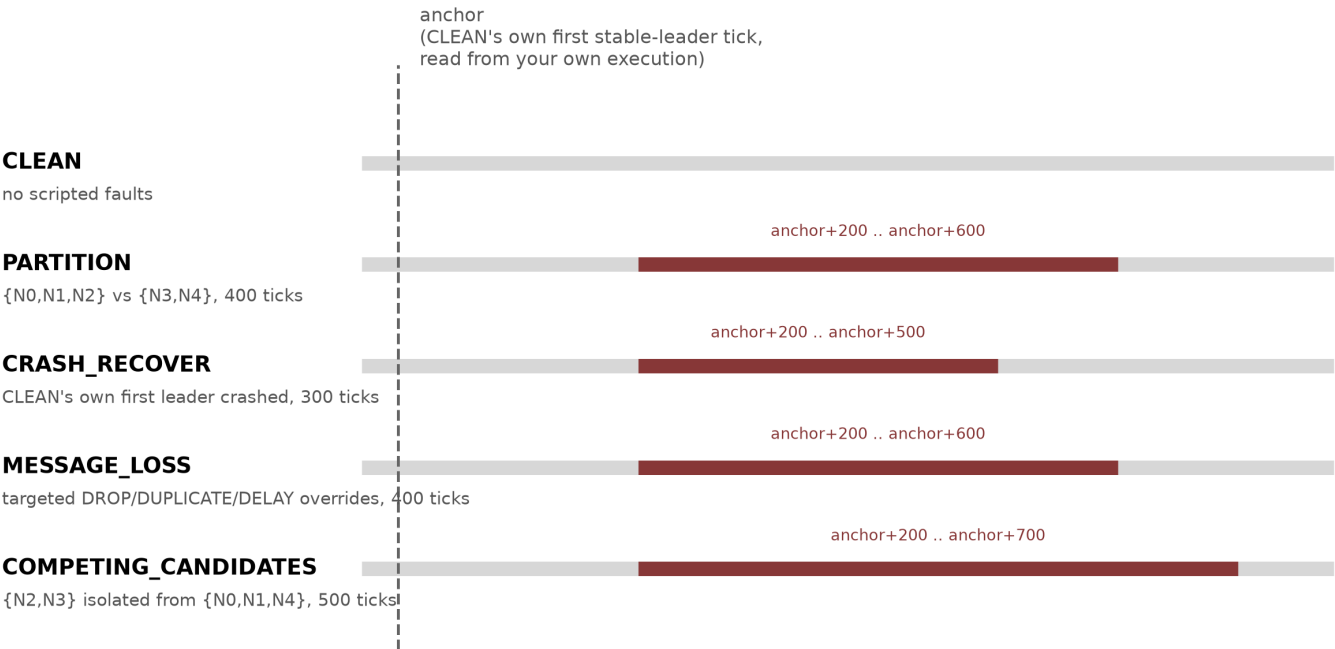


Figure 4 -- Per-scenario fault window, offsets relative to your own run's anchor tick.

## 1. CLUSTER AND TICK MODEL

5 nodes, N0-N4. Ticks are discrete integer time units. Each of the 5 named scenarios (item 8) is one continuous run of exactly its own stated 2400-tick length, run to completion -- never a shortened or sampled run. The full sweep is 3 election-timeout settings x 5 scenarios = 15 rows.

## 2. PER-NODE STATE

current\_term (int, persisted across crash), voted\_for (persisted, reset to None whenever current\_term increases), log (list of (term, cmd\_id) entries, 1-indexed, persisted), commit\_index (persisted). role (FOLLOWER/PRECandidate/Candidate/Leader, volatile, resets to FOLLOWER on crash recovery), election\_elapsed (volatile). While LEADER only: next\_index[j], match\_index[j] per other node j (volatile, reinitialized on becoming leader).

## 3. ELECTION TIMEOUT AND ITS RESET RULE (Figure 3)

Each node's own effective election timeout is  $\text{BASE\_TIMEOUT} + 20 \times \text{node\_id}$  ticks. BASE\_TIMEOUT is the swept setting: SHORT=150, MEDIUM=250, LONG=400. This per-node stagger is fixed and deterministic, never randomized. election\_elapsed resets to 0 only when: the node grants a vote; the node receives a valid AppendEntries at term  $\geq$  its own; or the node itself just began a fresh pre-vote round. It is not reset by merely receiving or sending a pre-vote message, and a LEADER does not track it.

## 4. PRE-VOTE PHASE (Figure 1)

On reaching its own effective timeout, a FOLLOWER (or a CANDIDATE after a failed election) becomes PRECANDIDATE without touching current\_term or voted\_for, resets election\_elapsed to 0, and sends a pre-vote request (term=current\_term+1, last\_log\_index, last\_log\_term) to every other node. A recipient grants a pre-vote iff the requester's log is at least as up-to-date as its own by the same log-completeness rule as a real vote (item 5) -- regardless of the recipient's own current\_term or voted\_for; granting a pre-vote never mutates voted\_for. On collecting grants from a majority ( $\geq 3$  of 5, itself included), the PRECANDIDATE proceeds to a real election (item 5). If its own effective timeout elapses again first without a majority, it stays PRECANDIDATE and resends a fresh pre-vote round, still without incrementing current\_term.

## 5. REAL ELECTION (Figure 1)

On winning a pre-vote majority: increment current\_term by 1, vote for self, become CANDIDATE, reset election\_elapsed, and request real votes. A recipient grants a vote iff: the request's term is  $\geq$  its own (adopting that term and stepping down first if strictly greater); its voted\_for is None or already the requester this term; AND the requester's log is at least as up-to-date (higher last\_log\_term, or equal and last\_log\_index  $\geq$ ). Concurrent same-tick requests from different candidates are evaluated in ascending requester-node-ID order. A CANDIDATE reaching a real-vote majority ( $\geq 3/5$ ) becomes LEADER immediately, initializing next\_index[j]=len(own log)+1 and match\_index[j]=0 for every other node j, and immediately starts replication (item 6). A split real vote (timeout without a majority) returns to a fresh pre-vote round, not a term increment.

## 6. LOG REPLICATION -- BOUNDED-BATCH APPENDENTRIES (Figure 2)

LEADER only. For each other node  $j$ , the leader sends an AppendEntries round to  $j$  every HEARTBEAT\_INTERVAL=10 ticks, or immediately after a new command is appended to its own log, whichever is sooner. The message carries prevLogIndex=next\_index[j]-1, prevLogTerm, up to  $K=4$  entries starting at next\_index[j] (a lagging follower needs multiple rounds to catch up), and leaderCommit=the leader's own commit\_index. A follower rejects iff prevLogIndex>0 and it lacks a matching entry there; on rejection the leader decrements next\_index[j] by 1 (linear backoff, no conflict-term optimization) and retries next round. On acceptance the follower truncates any conflicting existing entries the batch covers, appends the batch, and advances its own commit\_index toward leaderCommit if applicable; it reports its new matchIndex, and the leader sets match\_index[j] to that value and next\_index[j]=matchIndex+1.

## 7. COMMIT-INDEX ADVANCEMENT (LEADER)

The leader advances its own commit\_index to the highest index  $N$  such that a majority of nodes ( $\geq 3/5$ , itself included) have match\_index[j] $\geq N$  AND the log entry at index  $N$  has term == the leader's own CURRENT current\_term. An entry from an earlier term is never committed directly merely because a majority already has it -- only indirectly, once a same-term entry after it is committed by this rule.

## 8. NETWORK MODEL AND THE 5 NAMED SCENARIOS (Figure 4)

Every message sent at tick  $T$  is delivered at  $T+2$  ticks unless the active scenario's script overrides it: DELAY, DROP, or DUPLICATE for named message classes/pairs/windows. A partition window (a tick range plus a 2-way node split) drops every message between opposite sides for its whole duration. A crash window means the node sends and processes nothing at all for its whole duration; on recovery it resumes as FOLLOWER with election\_elapsed at 0, using its persisted state exactly as it was at the crash tick. Client commands are broadcast to all 5 nodes at fixed ticks; only whichever node is LEADER at that exact tick appends the command, every other node ignores it. The 5 named scenarios, each its own full 2400-tick run: CLEAN (no faults); PARTITION ({N0,N1,N2} vs {N3,N4}); CRASH\_RECOVER (CLEAN's own first elected leader crashed); MESSAGE\_LOSS (targeted DROP/DUPLICATE/DELAY overrides); COMPETING\_CANDIDATES ({N2,N3} isolated from the 3-node majority long enough that both independently exceed their own timeout repeatedly, demonstrating the pre-vote phase's purpose: 2 of 5 can never reach a 3-node pre-vote majority, so neither may ever increment current\_term or disrupt the majority once the partition heals).

## 9. THE SWEEP AND SELECTIONS

election\_timeout\_setting in {SHORT, MEDIUM, LONG} x the 5 named scenarios = 15 sweep rows, each a full run to completion. A setting is feasible only if it violates none of the three safety invariants (item 10) in any of its 5 runs. Among feasible settings, report two selections: recovery-optimal (minimizes ticks from CRASH\_RECOVER's own crash-window start to a new node establishing LEADER for a higher term) and overhead-optimal (minimizes total message count across all 5 scenarios combined). These need not agree; disclose explicitly whether they do.

## 10. INDEPENDENT VERIFICATION

A separately-coded verifier, sharing only immutable input constants with your primary implementation, must independently re-derive each scenario's full 5-node trace (or check a feasibility certificate) and confirm for every one of the 15 runs: election safety (no two nodes are LEADER for the same current\_term simultaneously); log matching (if two logs both hold an entry at the same index with the same term, every earlier index in both is identical); leader completeness (once any node commits an entry, every future leader has that exact entry at that index). Two required adversarial mutations: a vote granted to a candidate whose term is below the recipient's own current\_term; and a leader overwriting an already-committed log entry with a different one. The verifier must reject both while still accepting the true original.

## 11. CERTIFICATION

For the baseline configuration (election\_timeout=MEDIUM, scenario=PARTITION) only: serialize the complete per-tick trace as one line per tick "t|role0,term0|role1,term1|role2,term2|role3,term3|role4,term4" (role as its first letter), preceded by one header line "QUORUM7|MEDIUM|PARTITION", lines joined by \n, UTF-8 encoded, hashed with SHA-256; report the first 16 hex characters as the trace-integrity certificate.

Deliver: protocol-engine source, an independently-coded verifier, the full 15-row sweep table, a decision file with both selections and their agreement/divergence disclosure, the baseline trace plus its SHA-256 certificate and your adversarial-mutation results, and an engineering memo explaining the recovery/overhead trade-off and the pre-vote phase's role in the COMPETING\_CANDIDATES scenario. Base every reported number on your own executed implementation -- never hand-derive a trace, a sum, or the hash.